IcomRigControl — What This App Can Do (Capabilities Inventory)

A complete, plain-language inventory of everything IcomRigControl does today, plus what's planned but not built. Meant as a shared reference and a brainstorming springboard. Grounded in the real code as of 2026-08-15 (Phases 1–10 complete, 373 passing tests).

In a nutshell: IcomRigControl is a free, cross-platform (Windows/macOS/Linux/Raspberry Pi) control and logging program for the Icom IC-7300, IC-7300MK2, and IC-705 radios. It does everything Icom's own RS-BA1 remote software does *except* live remote audio — and adds a whole logging-and-operations ecosystem (QSO log, contests, callsign lookup, LoTW, HRD, N1MM/WSJT-X, APRS beaconing) that RS-BA1 has none of.

1. Radios & connection

Capability	Detail
Supported radios	Icom IC-7300 (CI-V address `0x94`) and IC-7300MK2 (`0xB6`)
Local connection	USB / serial CI-V, selectable COM port / `/dev/tty…`, 115200 baud default
Remote connection	TCP to a networked `CivTcpServer` (Phase 9) — token-authenticated
Demo mode	Full UI with a simulated radio, no hardware needed
IC-705	Implemented (address `0xA4`); meter scaling shares the IC-7300 decoder, pending real-hardware verification

2. Core radio control

- **Frequency** — read live from the radio and set it (BCD-encoded CI-V, verified against the reference).
- **Mode** — LSB, USB, AM, CW, RTTY, FM, CW-R, RTTY-R, DV — read and set.
- **PTT (transmit) control** — key the radio on/off from the app (CI-V `1C 00`), with the safety guarantees hardened in this session (never stuck on, never unidentified).
- **Remote power ON/OFF** — power the radio on/off from the app (CI-V `18 01` / `18 00`). Power-on sends a wake-up preamble and only works if the radio is in CI-V standby (matches RS-BA1's own limitation).
- **CW keyer & voice memories** — a Keyer window sends editable CW macros as Morse (CI-V `17`, up to 30 chars) and fires the radio's recorded voice memories T1–T8 (CI-V `28`).
- **Receive-only / TX-inhibit switch** — one master toggle that blocks *all* transmit (beacon, PTT, CW, voice, remote-audio TX) at the `Transceiver` level, with an unmissable red banner while active. For safe testing, or when another program (e.g. APRS-Command) owns transmit on the shared radio. Persists across restarts.
- **FT8 one-touch setup** — an "FT8 Setup" button puts the radio in USB-D + wide filter via CI-V (`26 00 01 01 01`), replicating the radio's FT8 preset (which isn't itself CI-V-accessible). WSJT-X still does the decode; NB/NR/AGC are left manual, as Icom's own preset does.
- **VFO / split** — VFO select, A=B, swap, split control commands are implemented in the engine.

- **Live meters, polled ~continuously:** S-meter (S-units + true dBm), RF power output %, SWR, ALC, supply voltage (Vd), current draw (Id).

3. Spectrum scope & waterfall (Phase 7)

- Live **spectrum scope** capture from the radio (`27 xx` commands).
- **Waterfall display** with a real **frequency axis** (labels in kHz/MHz).
- **Click-to-tune** — click a point on the waterfall and the radio tunes there.
- Configurable span.

4. Memory channel editor (Phase 4)

- **Bulk read** all 99 memory channels from the radio (skips empty ones), with progress.
- **Bulk write** channels back to the radio.
- Add/edit channels in a grid (frequency, mode).
- Cancelable long reads.

5. QSO logging — the resilient backup of record (Phase 8)

The core design principle: every contact lands in IcomRigControl's own local log, independent of any external program's health. Hardened this session so a QSO can never be lost to a write failure or a thread race.

- **Log a QSO** — auto-fills frequency, band, and mode from the live radio at the moment of logging.
- **Write-through persistence** — every QSO is immediately written to a timestamped ADIF session file (crash-safe).
- **ADIF export** — standard `.adi` output.
- **General + contest logging** in one UI, with a contest selector.
- **Contests supported (verified against official ARRL rules):** ARRL Field Day and ARRL RTTY Roundup — including exchange fields, dupe checking, serial numbers, and live score.

6. Callsign lookup (Phase 8c)

- Look up a callsign to auto-fill name and grid square while logging.
- **Sources:** QRZ.com (XML), HamQTH (XML), Callook.info (US) — user-selectable.
- Lookups are contractually non-throwing (never crash the logger); hardened with an extra guard this session.

7. Logbook of the World / LoTW (Phase 8d)

- **Upload** logged QSOs to ARRL LoTW (signing delegated to ARRL's own TQSL tool — never reimplemented).
- **Download confirmations** and mark confirmed QSOs (✓) in the log grid.

8. Ham Radio Deluxe integration (Phase 8e)

- One-way, best-effort bridge that writes QSOs into HRD Logbook's SQLite database (defensive, schema-checked). HRD being down never blocks the local log.

9. N1MM Logger+ / WSJT-X integration (Phase 8f)

- **Send** live rig state (frequency, mode, PTT) out as RadioInfo UDP packets, to one or more user-configured destinations (127.0.0.1 or explicit LAN).
- **Receive** logged contacts over UDP from N1MM/WSJT-X/HRD and fold them into the local log.

10. Activity logging (Phase 5)

- Continuous **CSV activity log** of rig state over time (timestamp, frequency, mode, all meter readings) for later analysis. Separate from the QSO log.

11. Remote / network operation (Phase 9)

- **Headless server mode** (`IcomRigControl.UI --headless-server`) — runs with no display, serves CI-V control to remote clients over TCP. Designed for a Raspberry Pi sitting next to the radio, reachable over LAN, VPN, or 44Net/AMPRNet.
- **Token authentication** required (never runs with a blank token).
- The desktop app can act as a **remote client** — from the app's perspective a remote radio is indistinguishable from a local one; every feature above works remotely.
- *Proven via loopback; real-hardware-over-real-network test still pending.*

12. HF APRS — beacon *and* receive (Phase 10 + HF-APRS build)

*Neither IC-7300 has a TNC — this is built as software AFSK/AX.25 packet audio. Note: IcomRigControl is the **only** HF APRS tool here — APRS-Command is VHF-only. HF APRS is also something Icom's RS-BA1 doesn't do at all.*

Transmit (beacon):

- Build an **APRS position beacon** (lat/lon, symbol, comment), auto-appending live frequency/mode.
- **AX.25 UI frame** + **AFSK modulation** (300-baud HF profile), now with a proper **FCS + HDLC flag preamble** so real receivers/igates can actually decode it (a latent bug fixed during the receive build).
- Keys PTT, plays the packet, releases PTT — with the session's safety fixes (guaranteed key-down, no cross-keying, callsign validated).
- **Manual** and **automatic** periodic beaconing.

Receive (decode):

- Full receive pipeline (the mirror of transmit): **AFSK demodulate** → **HDLC deframe (FCS-checked)** → **AX.25 decode** → **APRS parse** (uncompressed positions, text messages, status).
- **Live receive service**: listens on the radio's RX audio continuously, decodes with an automatic **bit-sync sweep** (catches packets no matter when they arrive), de-duplicates repeats, and never crashes on a bad decode.

- Proven end-to-end by both a modulate→decode **round-trip test** and a live-service test (no hardware needed).

HF APRS window (the station display):

- A **live station list** — one row per station heard, showing call, position, info/comment, and time last heard, updated in place as they beacon.
- A per-station **aprs.fi** button that opens that station on the aprs.fi map in your browser (the map, without building a map into the app — and yes, HF APRS *does* show up on aprs.fi, gatewayed by igates just like VHF).
- **Messaging**: a Messages panel for text messages addressed to you, plus a to-call + text **Send** box that transmits an APRS message over the same safety-gated path as the beacon.
- **Auto-ACK**: when someone sends you a numbered message, IcomRigControl can automatically reply with the expected acknowledgement — a one-click toggle turns it on or off (on by default). It never acknowledges an acknowledgement, so there are no loops.
- Opens as its own window from the **HF APRS** dashboard button, alongside the main radio panel (so you still watch the real TX indicator, meters, and TX-inhibit banner while it works) — it's a monitor + messaging panel, not a second set of radio controls.

13. DX Cluster & band map

- **Connect to a DX cluster** — pick from well-known nodes (NC7J, VE7CC, DXFun, Reverse Beacon Network for CW/RTTY and FT8/FT4) or enter a custom host/port; auto-logs in with your callsign. Connection details are saved.
- **Live spot list / band map** — incoming spots (time, kHz, DX call, comment, spotter) shown newest-first, capped so a busy node can't grow it without bound.
- **Click-to-tune** — a Tune button on every spot QSYs the radio to it.
- **Post your own spots** — announce a DX call to the cluster (`DX <kHz> <call> <comment>`), with a one-click "use radio frequency" prefill.
- Resilient: a dropped cluster connection is surfaced, never crashes the app.

14. CW decode & zero beat

*The radio decodes CW on its own screen but won't send that text out over CI-V (not even Icom's RS-BA1 can show it) — so IcomRigControl decodes Morse from the received **audio** itself.*

- **Live CW reader** — decodes Morse off the radio's receive audio into a scrolling text window, in real time.
- **Adaptive speed** — tracks the sender's speed (shown live in **WPM**) and follows a fist that speeds up or slows down; no manual speed setting.
- **Automatic-gain tone detection** — a Goertzel filter at your CW pitch with a floating threshold, so it copes with fading and level changes without a level knob.
- **Zero Beat button** — measures the actual pitch of the received tone, then tunes the radio (over CI-V) so it lands exactly on your CW pitch. That puts you precisely on the other station's frequency — "netted" — so they hear you where they expect. A **Reverse** toggle flips the tune direction for CW-Reverse.
- **Tuning hint** — tells you how far off (in Hz) and which way the signal sits, even before you press Zero Beat.
- Pitch and direction are saved. Proven by a modulate→decode **round-trip test** (no hardware needed).

- **Save button** — writes the decoded text to a dated `.txt` file (Documents → IcomRigControl → Decoded) you can open and **print**.
- *Honest note:* like every software CW reader, it copies clean and machine-sent CW very well and struggles with sloppy sending, deep fading, or interference — a good ear still wins there.

15. RTTY decode

RTTY (radioteletype) is a defined machine mode, so — unlike CW — it decodes very reliably once you're tuned in.

- **Live RTTY reader** — decodes standard HF Baudot RTTY (45.45 baud, 170 Hz shift, 2125/2295 Hz tones) off the receive audio into a scrolling text window, in real time.
- **Automatic letters/figures handling** — follows the Baudot shift codes, so digits and punctuation come out right.
- **Reverse button** — swaps mark and space for a signal on the wrong sideband (the classic RTTY "why is it garbage" fix); takes effect immediately.
- **Save button** — writes the decoded text to a dated `.txt` file (Documents → IcomRigControl → Decoded) that you can open and **print**.
- Proven by a modulate→decode **round-trip test**, including a figures/numbers message and a reversed-polarity signal.

16. Phone / tablet remote (browser control)

Operate the radio from any phone, tablet, or laptop browser on your network — nothing to install on the device. This goes beyond Icom's RS-BA1, which needs its own software on the remote computer.

- **Open a web page, control the radio.** IcomRigControl serves a small mobile web page; open its address (e.g. `http://192.168.1.50:8080`) in any browser and you get a live control panel — big frequency readout, mode buttons, S-meter and power/SWR/ALC/voltage, tuning buttons, direct frequency entry, and a PTT button.
- **Live and two-way** — the page updates about five times a second and sends your tuning/mode/PTT back to the radio in real time.
- **Runs from your PC or from a Raspberry Pi at the radio.** On the desktop, a **Phone / Tablet** button lists the exact addresses to type on your phone. On a headless Pi, add `--webport 8080` and browse to the Pi.
- **Live waterfall, with tap-to-tune** — the spectrum waterfall streams to the phone too; tap anywhere on it to jump the radio to that frequency.
- **Listen to the radio** — tap **Listen** and the receive audio streams to your phone/tablet and plays in the browser (over the network — no Bluetooth, no app). Works alongside the meters and waterfall.
- **Talk (transmit your voice)** — a **Hold to Talk** button keys the radio and sends your phone's microphone audio to transmit; releasing it unkeys. Only one person can transmit at a time, transmit always obeys the Receive-Only / TX-inhibit switch, and the radio is never left keyed. Because browsers only allow the microphone over a secure connection, turn on **Use HTTPS** (one checkbox) and Talk works from a phone over your network — your browser shows a one-time "not trusted" warning you accept (the certificate is self-signed by the app).
- **Nothing to install, works offline** — the page is completely self-contained (no internet, no app store), so it works on an isolated field network.

- **Safe by design** — an optional access **token** gates it, transmit always obeys the Receive-Only / TX-inhibit switch, and you can enable **HTTPS** to encrypt the link. Intended for a trusted home network or a VPN.

17. Band DVR (record & rewind the radio)

A DVR for the airwaves — catch the call you just missed, or record a whole session.

- **Instant replay** — while monitoring, the last 60 seconds of receive audio are always kept in memory; tap **Replay last 30s** or **60s** to hear it again ("who was that? — rewind").
- **Record to a file** — flip **Record** to capture the receive audio continuously to a standard **WAV** file (Documents → IcomRigControl → Recordings), openable in any media player or editor.
- **Cross-platform, no extra software** — the recorder is built in, and works on Windows, macOS, Linux, and the Raspberry Pi.
- Opens in its own window from the **Band DVR** dashboard button, alongside the radio.
- *Planned next:* recording/scrubbing the **waterfall**, and auto-attaching each contact's audio to its log entry.

18. Settings & platform

- One Settings window covering connection, APRS, callsign lookup, LoTW/TQSL, HRD, and N1MM.
- Cross-platform audio: playback and capture on Windows (NAudio/WASAPI), Linux/Raspberry Pi (ALSA `arecord`/`aplay`), and macOS (`afplay` for beacons, plus SoX `rec`/`play` for the live decoders and remote audio — macOS needs `brew install sox`).
- Settings persisted to JSON; secrets kept out of source control.
- **Windows, macOS, Linux desktop, and Raspberry Pi OS** all supported via Avalonia.

19. Architecture (for context)

Four clean layers: **CivEngine** (CI-V framing, serial I/O, APRS/AFSK, CW & RTTY DSP) → **RigModel** (`Transceiver` + the network layer) → **Services** (all the integrations above, incl. the web remote server and Band DVR) → **UI** (Avalonia views/view-models). 388 automated tests.

What it does NOT do yet — the brainstorm springboard

These are the natural places to take it next. Items marked *(documented)* already have notes in CLAUDE.md.

Small / near-term

- ~~Remote Power ON/OFF~~ — **DONE** (implemented 2026-08-15; CI-V `0x18`, Power On/Off buttons on the dashboard).
- **Real-hardware validation** of Phase 9 remote mode over an actual network link.
- **Meter scaling verification** — Po/ALC/scope-span byte layouts need confirming against a real radio (flagged in the audit).

Medium

- **IC-705 real-hardware verification** — support is now implemented (first pass: enum + address `0xA4` + Settings picker). The remaining step is confirming meter scaling byte-for-byte against the real radio. (GPS/D-PRS and D-STAR remain out of scope.)
- **UI redesign** *(documented)* — you've said the current UI isn't satisfying; this is the gate for the comprehensive User Manual. Open questions: color/theme, transceiver-panel vs. flat dashboard, tabs vs. one scrolling window.
- **Phase 11: clickable radio front-panel** *(documented)* — a photo/vector of the rig with clickable controls.

Large / its own project

- **Phase 12: Remote Audio** — **CODE-COMPLETE** (live hardware tuning pending). UDP + Opus + full duplex, cross-platform incl. Raspberry Pi. Built and tested: Opus codec (pure-C# Concentus), UDP packet + jitter buffer, full platform audio I/O (Windows/NAudio, **Pi/Linux ALSA**, macOS stub), continuous stream output, the full-duplex `RemoteAudioLink` engine (verified end-to-end over a real socket), a **Remote Audio window** with a push-to-talk Transmit toggle that keys the radio's PTT over the CI-V link, and the **headless Pi server** streaming radio audio via `--audioport`. (Bonus: the Linux player also makes the APRS beacon work on Linux/Pi.) Only real-hardware latency tuning remains — the live testing session.
- **Comprehensive User Manual** *(documented)* — real screenshots, click-by-click, every quirk; triggered once the UI-redesign question is resolved.

Decided / in progress (2026-08-15)

- ~~CW keyer / voice memories~~ — **DONE** (Keyer window; CI-V 17 / 28).
- ~~DX Cluster + band map~~ — **DONE** (section 13): multi-cluster picker, live spot list, click-to-tune, and spot posting. A waterfall spot-overlay is a possible future enhancement.
- **Rotator / antenna switch / amplifier control** — **not planned**: not CI-V functions, and no test hardware on hand (would ship unverifiable support).
- **Digital modes** — FT8 stays **WSJT-X only** (no in-app FT8 — reimplementing it isn't sane); an **"FT8 Setup" button** now replicates the radio's FT8 preset (USB-D + wide filter) over CI-V as a convenience. In-app **RTTY and CW decode** are feasible "path 2" features but are **gated on Phase 12 audio capture**, so deferred until after the audio work. WSJT-X integration is untouched.
- More contests; general-logging niceties (QSL cards, statistics, maps).
- Mobile/tablet or web front-end to the headless server.